

Machine Learning 1

"field of study that gives computers the ability to learn without being explicitly programmed"

ML Algorithms

- Supervised Learning

We have labelled data (input-output pairs), we suspect a structure that maps them and use it to predict outputs for new inputs.

- Unsupervised Learning

We have many inputs-outputs but have no sense of structure, we use models to learn it in different ways.

- Recommender Systems

- Reinforcement Learning

Supervised Learning

$$\begin{array}{ccc} X & \rightarrow & Y \\ \text{input label} & & \text{output label} \end{array}$$

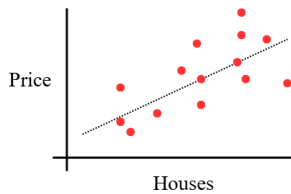
- Learns from being given right answers ?

Example

Spam-filtering: Email genuine or spam ?

Example

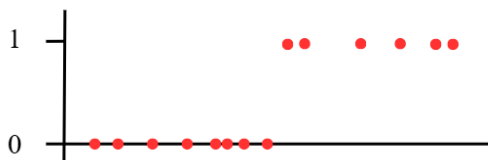
Regression in Housing Price Prediction



Regression: tries to predict numbers from ∞ possibilities.

Example

Classification: breast cancer (malignant or benign ?)

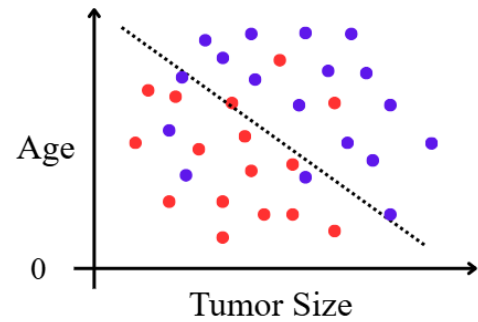


Classification: only finite outputs/categories

Example

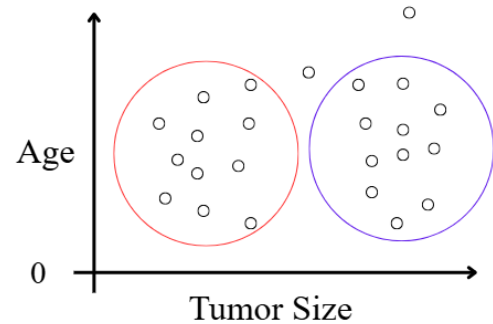
We can have more than a single input (previous example :

size), we can have many dimensions. (e.g., 2)



Example

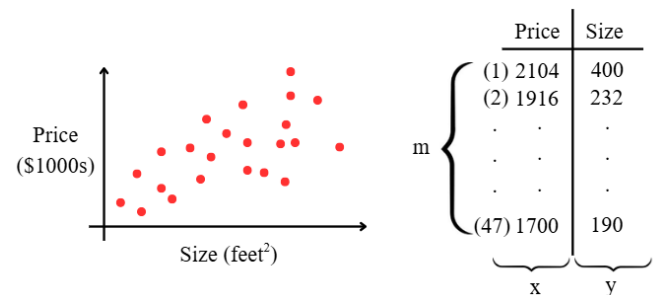
Unsupervised Learning: The following is an example of unsupervised learning being applied; non-labelled data is discriminated (clustering algorithm)



Another example of clustering algorithm being applied is *Google News*: clusters news articles together from the internet. Other applications can include: anomaly detection, dimensionality reduction, etc.

Linear Regression with One Variable

Example



Initial Terminology

- **m**: number of training examples
- **x**: "input-feature" / "feature"
- **y**: "target-feature" / "output"
- **(x, y)**: a single training example

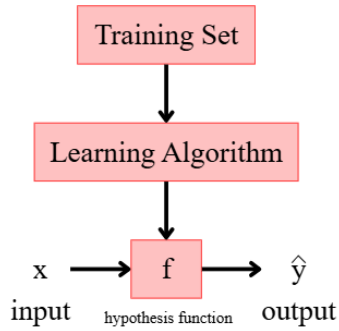
Example

$(2104, 400) = (x^{(1)}, y^{(1)})$ is a specific example.

More generally, $(x^{(i)}, y^{(i)})$ refers to a specific row in the table, i.e., the i^{th} training example.

Note: $x^{(2)} \neq x^2$ i.e., it is not exponentiation.

Typical Training of Supervised ML models



Note: the hypothesis function is also called "the model".

Note: The smaller the $|y - \hat{y}|$ the better we did.

How to represent f ?

Note: Linear Regression: A straight line

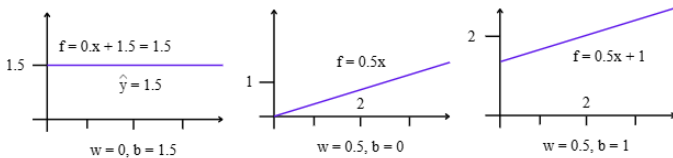
Model: $f_{w,b}(x) = wx + b$

Note: w and b are called **parameters** of the model / **coefficients** / **weights**

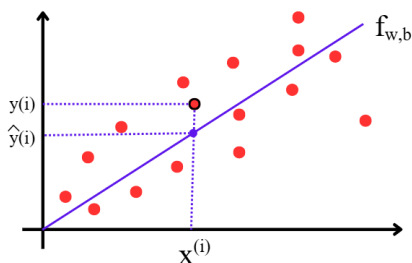
Note: Linear Regression with one variable (univariate) means only 1-input feature: x

Example

We can by trial and error plot all kinds of lines,



Mathematical Setup



$$\hat{y}^{(i)} = f_{w,b}(x^{(i)})$$

$$f_{w,b}(x^{(i)}) = wx^{(i)} + b$$

Goal: find w, b so that $y^{(i)}$ is as close to $\hat{y}^{(i)} \forall (x^{(i)}, y^{(i)})$

Measuring Cost: To measure how far are we from a perfect model ?

$$\frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

- error is squared, as all positive values are easier to deal with (negative might cancel out positives making errors seem smaller than they actually are)
- $\frac{1}{m}$ is to get an average cost/error
- To make the derivative of this function simpler, we can have a 2 in division so that it cancels the 2 that will come from the chain rule coefficient of the derivative.

Squared Error Cost Function:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Note: Most commonly used for regression problems.

Univariate Linear Regression Setup So far

- Model

$$f_w(x) = w.x$$

- Parameters

$$w$$

- Cost Function

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_w(x^{(i)}) - y^{(i)})^2$$

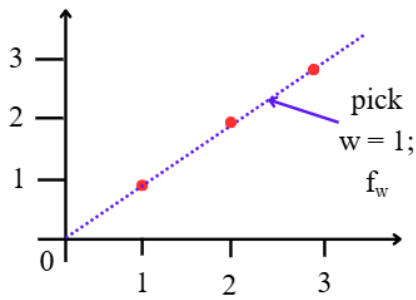
- Goal

$$\underset{w}{\text{minimise}} J(w)$$

Example

Consider $f_w(x)$ values,

for fixed w, $f_w(x)$ becomes function of f (in an example where we have 3 examples).



$w=1$:

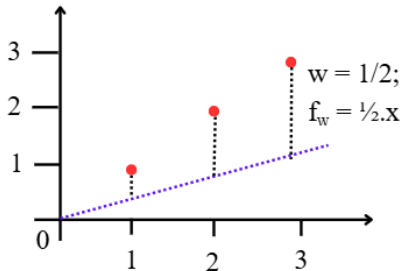
$$J(w) = \frac{1}{2m} \sum_{i=1}^m (f_w(x^{(i)}) - y^{(i)})^2$$

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} - y^{(i)})^2$$

For $w = 1$: $f(x) = y$

$$= \frac{1}{2m} (0^2 + 0^2 + 0^2) = 0$$

Consider another value,



$$J\left(\frac{1}{2}\right) = \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} - y^{(i)})^2$$

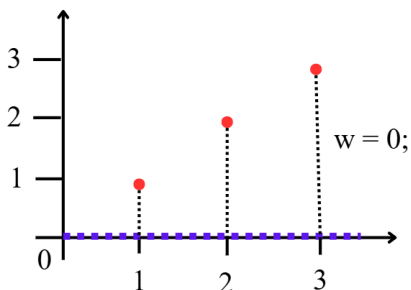
$$= \frac{1}{2m} \sum_{i=1}^m \left(\frac{x^{(i)}}{2} - y^{(i)}\right)^2$$

$$= \frac{1}{6} [(0.5 - 1)^2 + (1 - 2)^2 + (1.5 - 3)^2]$$

$$= \frac{1}{6} [0.25 + 1 + 2.25]$$

$$\frac{3.5}{6} = 0.58$$

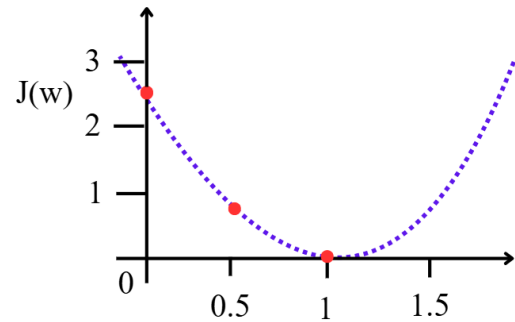
Let us consider another one,



$$J(0) = \frac{1}{2m} (1^2 + 2^2 + 3^2)$$

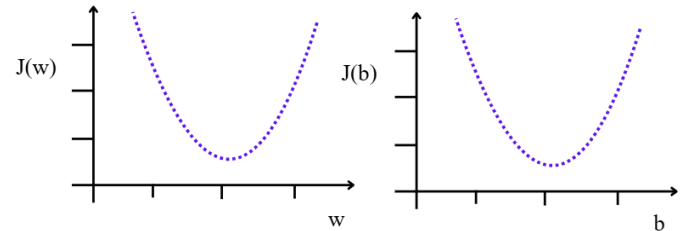
$$= \frac{1}{6} (1 + 4 + 9) = \frac{14}{6} \approx 2.3$$

For these values, $J(w)$ produces the following curve,

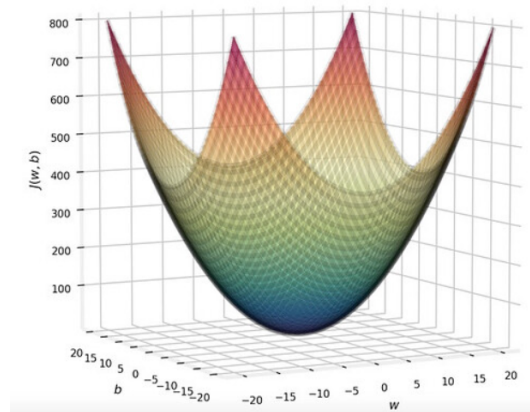


We notice that, $J(\frac{1}{2}) \approx 0.58$, $J(1) = 0$ and $J(0) \approx 2.3$. i.e., we have achieved the goal of minimising $J(w)$ at $w = 1$.

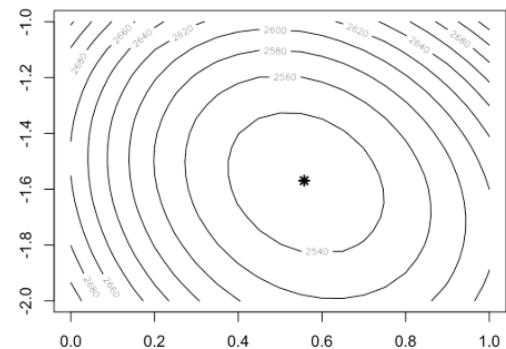
Had we extended this idea to 2 variables we would have to do something like this,



$J(w, b)$



This can also be visualised as a contour plot,



Note: The closer the point is to the centre, the more close it is to the global minimum.

The algorithm that helps us do this is called **Gradient Descent**

Gradient Descent

We have: $J(w, b)$

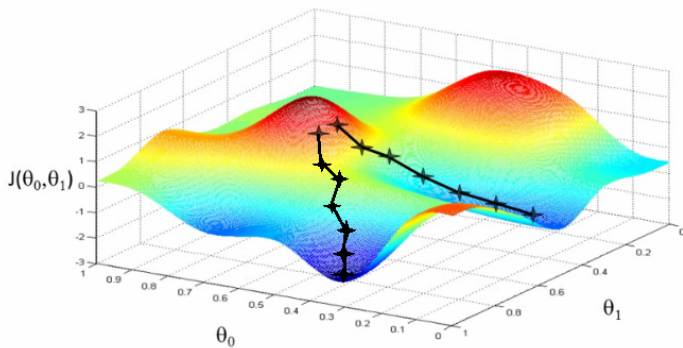
We want: $\min_{w, b} J(w, b)$ or

more generally, $\min_{w_1, w_2, \dots, w_n, b} J(w_1, w_2, \dots, w_n, b)$.

Outline:

- random w, b
- Keep changing w, b to reduce $J(w, b)$ until we hit minimum.

We may have other models that give us non-convex shaped Cost-functions, where at each step we look around and take a step in the direction of the steepest descent.



Note: In the above example we have two different possible end points, and based on our start points we might end up at different local minima(s)

Note: For Linear Regression we are guaranteed to get a bowl /convex shaped Cost function (in all dimensions) i.e., a single global minimum.

Gradient Descent Algorithm

until convergence ...

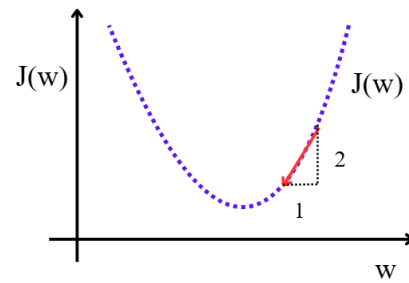
$$\begin{cases} w \triangleq w - \alpha \cdot \frac{\partial}{\partial w} J(w, b) \\ b \triangleq b - \alpha \cdot \frac{\partial}{\partial b} J(w, b) \end{cases}$$

where α is the learning rate and we simultaneously update w and b .

Correct Simultaneous Update

$$\begin{cases} w_{top} = w - \alpha \frac{\partial}{\partial w} J(w, b) \\ b_{top} = b - \alpha \frac{\partial}{\partial b} J(w, b) \\ w = w_{top} \\ b = b_{top} \end{cases}$$

Note: Doing the $w = w_{top}$ after the $b_{top} = b - \alpha \frac{\partial}{\partial b} J(w, b)$ is super important, cause the value of $J(w, b)$ will until the second step

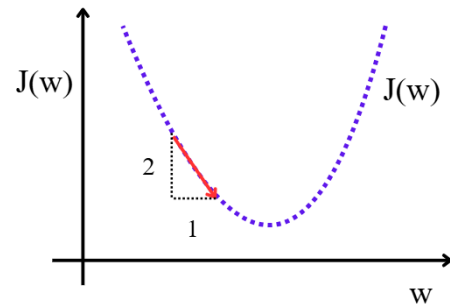


$$w \triangleq w - \alpha \left[\frac{d}{dw} J(w) \right]$$

where $\frac{d}{dw} J(w) > 0$ (positive slope),

$$w \triangleq w - \alpha \cdot (\text{positive value})$$

meaning the point will move leftwards (reduced)



$$w \triangleq w - \alpha \left[\frac{d}{dw} J(w) \right]$$

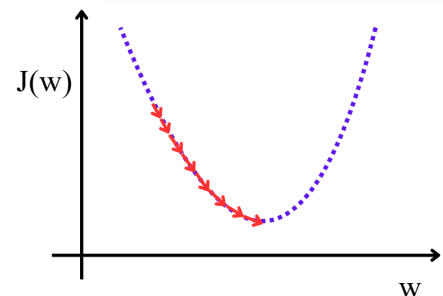
where $\frac{d}{dw} J(w) < 0$ (negative slope),

$$w \triangleq w - \alpha \cdot (\text{negative value})$$

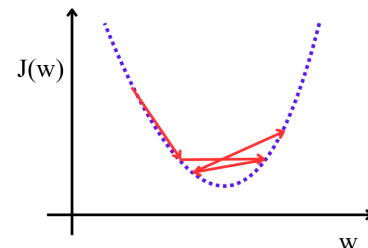
meaning the point will move rightwards (increased)

Effect of choice of α (Learning Rate)

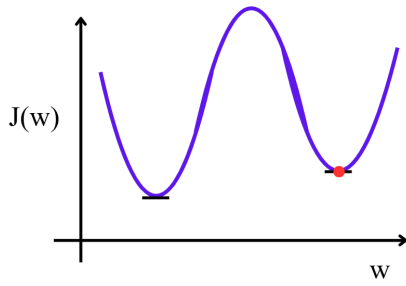
$$w \triangleq w - \alpha \frac{d}{dw} J(w)$$



Note: If α is too small, we will need more steps to get to minimum point of $J(w)$.



Note: If α is too large, we may overshoot $J(w)$ and get oscillating behaviour (may fail to converge).

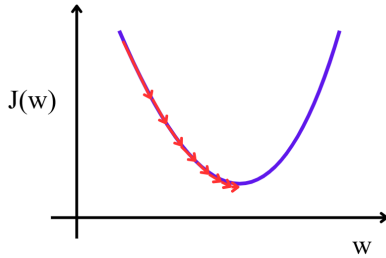


Also,

Note: it is possible for the gradient descent algorithm to reach local minima(s) (in situations where this is possible). This happens because $\frac{d}{dw}J(w)$ becomes smaller the lesser the slope (the closer we are to the minima).

$$\begin{aligned} w &\triangleq w - \alpha \frac{d}{dw}J(w) \\ \text{slope} &= 0 \\ \implies w &\triangleq w - \alpha \cdot 0 \\ w &\triangleq w \text{ (unchanged)} \end{aligned}$$

Moreover,



Note: Near a local minimum derivatives become smaller, i.e., we can reach minimum without decreasing learning rate α

Summary of the Linear Regression Setup so far

Linear Regression Model:

$$f_{w,b}(x) = wx + b$$

Cost Function:

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient Descent Algorithm:

repeat until convergence:

$$\begin{cases} w \triangleq w - \alpha \cdot \frac{\partial}{\partial w}J(w,b) \\ b \triangleq b - \alpha \cdot \frac{\partial}{\partial b}J(w,b) \end{cases}$$

where the derivatives are,

$$\begin{aligned} \frac{\partial}{\partial w}J(w,b) &= \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \cdot x^{(i)} \\ \frac{\partial}{\partial b}J(w,b) &= \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \end{aligned}$$

The derivations for the same are as follows,

$$\begin{aligned} \frac{\partial}{\partial w}J(w,b) &= \frac{\partial}{\partial w} \cdot \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x) - y^{(i)})^2 \\ &= \frac{1}{2m} \sum_{i=1}^m \frac{\partial}{\partial w} (w \cdot x^{(i)} + b - y^{(i)})^2 \\ &= \frac{1}{2m} \sum_{i=1}^m (w \cdot x^{(i)} + b - y^{(i)}) \cdot (x^{(i)} + 0 + 0) \cdot 2 \end{aligned}$$

$$\frac{\partial}{\partial w}J(w,b) = \frac{1}{m} \sum_{i=1}^m (w \cdot x^{(i)} + b - y^{(i)}) \cdot x^{(i)}$$

Hence the $\frac{1}{2}$

$$\begin{aligned} \frac{\partial}{\partial b}J(w,b) &= \frac{\partial}{\partial b} \cdot \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x) - y^{(i)})^2 \\ &= \frac{1}{2m} \sum_{i=1}^m \frac{\partial}{\partial b} (w \cdot x^{(i)} + b - y^{(i)})^2 \\ &= \frac{1}{2m} \sum_{i=1}^m 2 \cdot (w \cdot x^{(i)} + b - y^{(i)}) \cdot (1) \end{aligned}$$

$$\frac{\partial}{\partial b}J(w,b) = \frac{1}{m} \sum_{i=1}^m (w \cdot x^{(i)} + b - y^{(i)})$$

Final Gradient Descent Algorithm

repeat until convergence

$$\begin{cases} w \triangleq w - \alpha \frac{1}{m} \sum_{i=1}^m f_{w,b}(x^{(i)} - y^{(i)}) \cdot x^{(i)} \\ b \triangleq b - \alpha \frac{1}{m} \sum_{i=1}^m f_{w,b}(x^{(i)} - y^{(i)}) \end{cases}$$

Note: Each step of Gradient Descent, uses all the training examples (to calculate the cost), this is called **Batch Gradient Descent**.

Multiple Features (variables) / Multiple Linear Regression

Note: multiple linear regression is not the same as multivariate regression.

Example:

Size (feet) ²	Number of bedrooms	Number of toilets	Age of home (years)	Price (\$1000s)
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
x_1	x_2	x_3	x_4	y

Updated Notation

- $x^j = j^{th}$ feature.
- x = number of features.
- $\vec{x}^{(i)}$ = feature of i^{th} training example.
- $x_j^{(i)}$ = value of j^{th} feature of the i^{th} training example.

Hypothesis Function:

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

where:

- $\vec{w} = [w_1, w_2, \dots, w_n]$: row vector
- b is a number
- $\vec{x} = [x_1, x_2, \dots, x_n]$

All of this can be represented with a dot product,

$$f_{\vec{w},b}(\vec{x}) = \underbrace{\vec{w} \cdot \vec{x}} + b = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Note: Vectorised code is about writing code in such a way fashion so that its much faster at the execution level (as computers have many provisions for computing vector operations faster than a long list of regular operations).

i.e., instead of

$$f_{\vec{w},b}(\vec{x}) = \sum_{j=1}^n w_jx_j + b$$

we should be doing,

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

Dot products / vector operations are optimised instead of for loops (such as `np.dot(a, b)`)

Multiple Regression with Vector Notation

parameters:

$$b, \vec{w} = [w_1, w_2, \dots, w_n]$$

Model:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

Cost:

repeat {

$$w_j \triangleq w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b \triangleq b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

}

Gradient Descent for Multiple Features

for n features ($n \geq 2$):

repeat {

$$j = 1 : w_1 \triangleq w_1 - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \cdot x_1^{(i)}$$

$\vdots$

$$j = n : w_n \triangleq w_n - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \cdot x_n^{(i)}$$

$$b \triangleq b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

simultaneously update:

$$w_j \quad \forall j = 1, \dots, n \text{ \& } b$$

}

An alternate to Gradient Descent

Normal Equation: Works only for Linear Regression, no iterations needed.

Disadvantages

- Doesn't generalise to other learning algorithms
- Slow when features ($>10,000$)

$$\begin{bmatrix} b \\ w_n \\ \vdots \\ w_1 \end{bmatrix} = (\vec{x}^T \vec{x})^{-1} \vec{x}^T y$$

Note: computing $(X^T X)^{-1}$ is $O(n^3)$, so it gets very slow when you have many features. Gradient descent scales much better.

Feature Scaling

Note: Makes Gradient Descent run faster.

Example:

$$\widehat{price} = w_1x_1 + w_2x_2 + b$$

where x_1 is the size (feet)² [300-2000] and x_2 is the number of bedrooms [0-5].

Example house,

$$x_1 = 2000, x_2 = 5, price = \$500k$$

Consider,

$$w_1 = 50, w_2 = 0.1, b = 50$$

Then,

$$\widehat{price} = 50 * 2000 + 0.1 * 5 + 50 = \$100,050.5 \text{ (far from actual)}$$

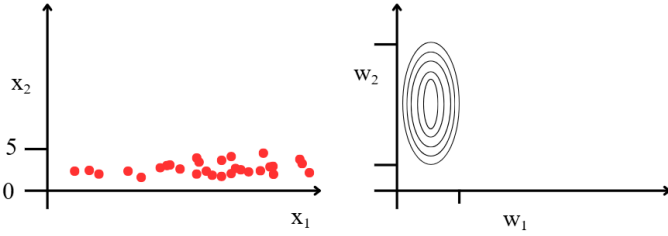
Consider,

$$w_1 = 1, w_2 = 50, b = 50$$

Then,

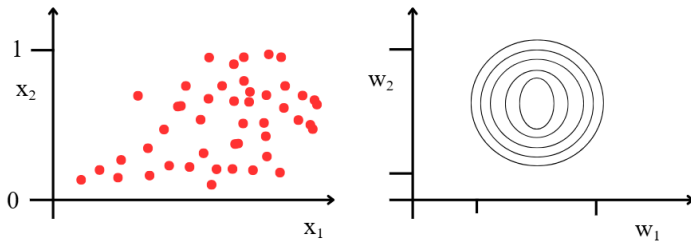
$$\widehat{price} = 0.1 * 2000 + 50 * 5 + 50 = \$500k \text{ (good estimate)}$$

Note: When the range of a feature is large a model will try to learn a small value for this, 0.1 for x_1 (bedrooms). Because of this we have curve that look like the following,



Note: Notice how we get thin ovals in the contour plot since w_1 has large effect from small changes as well ! For this reason Gradient Descent will bounce and struggle too much.

To fix this we can rescale inputs (normalise) on a standard 0 to 1 scale (or of the same order) for all inputs,



Different approaches to scaling

– Max-Division

Example:

$$\begin{aligned} 300 \leq x_1 \leq 2000, & \quad 0 \leq x_2 \leq 5 \\ x_{1,scaled} = \frac{x_1}{2000}, & \quad x_{2,scaled} = \frac{x_2}{5} \\ 0.15 \leq x_{1,scaled} \leq 1, & \quad 0 \leq x_{2,scaled} \leq 1 \end{aligned}$$

– Mean Normalisation

Example:

Find μ_{x_1} (mean),

$$x_{1,mean-normalised} = \frac{x_1 - \mu_1}{2000 - 300}$$

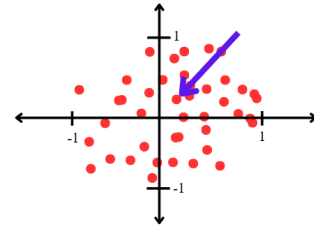
and

$$x_{2,mean-normalised} = \frac{x_2 - \mu_2}{5 - 0}$$

we get,

$$-0.18 \leq x_1 \leq 0.82, \quad -0.46 \leq x_2 \leq 0.54$$

The following graph shows what mean normalisation achieves,



– Z-score Normalisation:

Example:

$$x_{1,z-s-n} = \frac{x_1 - \mu_1}{\sigma_1}, \quad x_{2,z-s-n} = \frac{x_2 - \mu_2}{\sigma_2}$$

where σ_1 and σ_2 are the standard deviations of x_1 and x_2 respectively.

$$-0.67 \leq x_{1,scaled} \leq 3.1, \quad -1.6 \leq x_{2,scaled} \leq 1.9$$

Tips for Feature Scaling

- Aim for about $-1 \leq x_j \leq 1$ for each feature x_j , but -1 and 1 is generally loose.

Example:

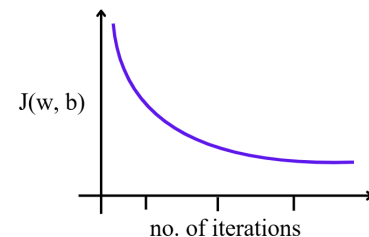
$-0.3 \leq x_j \leq 0.3$ is fine or $-3 \leq x_j \leq 3$ is also fine.

Example:

$-0.001 \leq x_j \leq 0.001$ is too small, and could benefit from rescaling.

Validating Gradient Descent

Number of iterations for Gradient Descent to minimise cost Function varies,



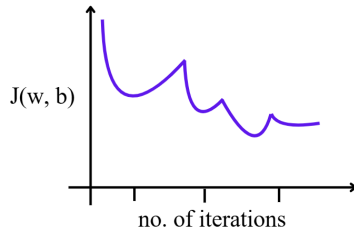
Arithmetic Convergence Test

- Let " ϵ " be 10^{-3}
- If $J(\vec{w}, b)$ decreases by $\leq \epsilon$ in one iteration, declare convergence.

Note: Choosing ϵ is tricky, better to look at curves.

Example:

Say we discover the behaviour of our Cost Function to be something like the following,

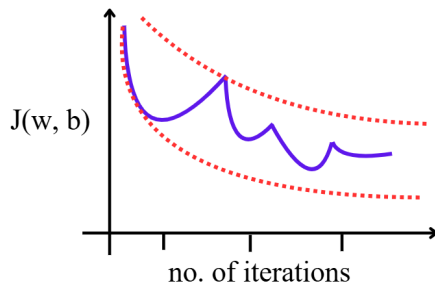


This would suggest that either Gradient Descent is not working or α is too large.

Tip: A good idea in such a situation is to try α (s) of different orders

..., 0.001, 0.01, 0.1, ...

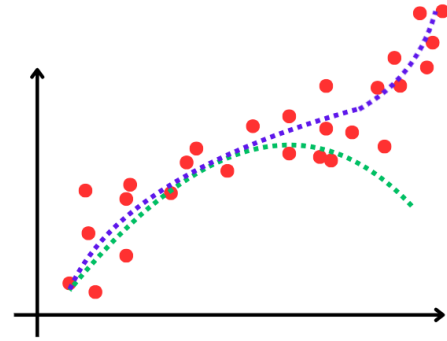
Using these we can get envelopes of where α is too large or where α is too small.



Once we have the order, we can change magnitude to fit a better value.

Polynomial Regression

Example:



Quadratic could have been good but it will dip in the end, so cubic might be better.

Note: When considering

$$f_{\vec{w},b}(\vec{x}) = w_1x + w_2x^2 + w_3x^3 + b$$

feature scaling becomes important as now have squared/cubic terms! Values will blow up ($1-10^3$), ($1-10^6$), ($1-10^9$) so we need to feature scale.

Example:

We can also have statements such as,

$$w_1x + w_2\sqrt{x} + b$$

Feature Engineering

Using Intuition/design we can create new features by transforming/combining original features.

Example:

For a house, a Model could have 2 variables:

- Frontage of house x_1
- Depth x_2

$$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + b$$

But we could make the observation that the area of the house has great relevance as well, giving us a 3rd feature,

$$x_3 = x_1 \cdot x_2$$

$$\implies f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

Classification

Example:

Binary (2 classes) classification can be of use in many situations

- Tumour: benign or malignant ?
- Email: Spam or not ?

Often the result is represented as 0 (negative class) or 1 (positive class)

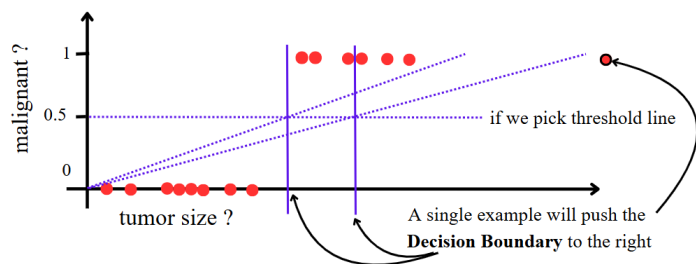
Intuition:

In Classification we usually come up with a continuous output that we split on a threshold, so for the tumour example, if our model says that probability is 0.6 we would say 1 (tumour)

$$\begin{cases} f_{w,b}(x) < 0.5 \rightarrow \hat{y} = 0 \\ f_{w,b}(x) > 0.5 \rightarrow \hat{y} = 1 \end{cases}$$

The reason why a regressive model like the ones we have been seeing isn't a good idea is because, it is highly sensitive to the extreme magnitude of examples and their volume as well, if the majority of examples are of label = 1 (tumour) the regression line will push the decision boundary

un-usefully far,



To solve this problem we introduce Logistic Regression

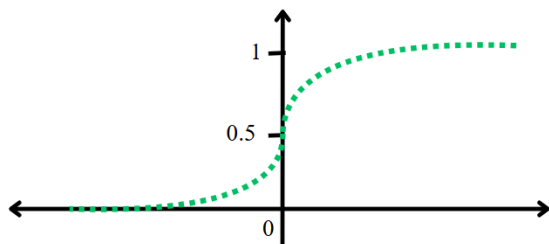
Logistic Regression

Note: The mention of the term "Regression" in the title is most misleading, as it has nothing to do with regressive tasks.

For such a sharp delineation we can use a function such as the Logistic function,

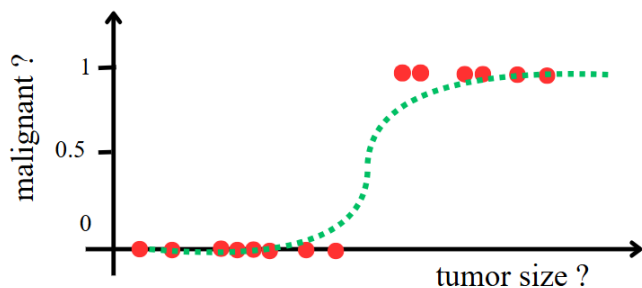
$$g(z) = \frac{1}{1 + e^{-z}}, \quad 0 < g(z) < 1$$

which looks like,



Example:

This is also a good fit for our previous example,



Note:

- $g(0) = 0.5$
- $g(\text{large positive number}) \approx 1$
- $g(\text{large negative number}) \approx 0$

We also see,

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

Also we know that,

$$P(y=0) + P(y=1) = 1$$

So, we can say,

$$f_{\vec{w},b}(\vec{x}) = P(y=1|\vec{x}; \vec{w}, b)$$

this is read as,

"Probability that y is 1, given input $\vec{x}$, parameters $\vec{w}$, b"

Setting Threshold

$$f_{\vec{w},b}(\vec{x}) \geq 0.5$$

$$\text{Yes} \Rightarrow \hat{y} = 1 \quad \text{No} \Rightarrow \hat{y} = 0$$

$$\text{When is } f_{\vec{w},b}(\vec{x}) \geq 0.5?$$

$$g(z) \geq 0.5$$

$$z \geq 0$$

$$\vec{w} \cdot \vec{x} + b \geq 0 \quad \vec{w} \cdot \vec{x} + b < 0$$

$$\hat{y} = 1$$

$$\hat{y} = 0$$

Decision Boundaries

$$f_{\vec{w},b}(\vec{x}) = g(z) = g(w_1x_1 + w_2x_2 + b)$$

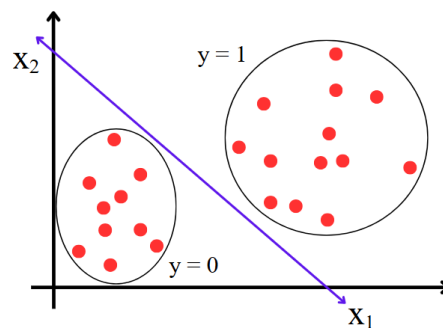
Example:

Say $w_1 = 1$, $w_2 = 1$ and $b = 3$ then, this line is called decision boundary

$$z = \vec{w} \cdot \vec{x} + b = 0$$

$$\Rightarrow z = x_1 + x_2 - 3 = 0$$

$$\Rightarrow x_1 + x_2 = 3$$



Non Linear Decision Boundaries

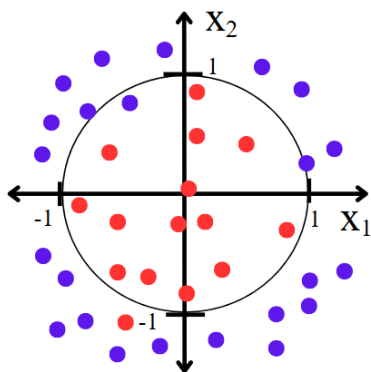
Example:

$$f_{\vec{w},b}(\vec{x}) = g(z) = g(w_1x_1^2 + w_2x_2^2 + b)$$

Say $w_1 = 1$, $w_2 = 1$ and $b = -1$

$$\text{Decision Boundary: } z = x_1^2 + x_2^2 - 1 = 0$$

$$\Rightarrow x_1^2 + x_2^2 = 1$$



$$\text{if } x_1^2 + x_2^2 \geq 1 \rightarrow \hat{y} = 1$$

$$\text{if } x_1^2 + x_2^2 \leq 1 \rightarrow \hat{y} = 1$$

Note: We can have even more complex Decision Boundary

Example:

$$f_{\vec{w},b}(\vec{x}) = g(z) = g(w_1x_1 + w_2x_2 + w_3x_1^2 + \dots + w_5x_2^2 + w_6x_1^3 + b)$$

Cost Function

Developing Terminology

Example:

Consider,

	x_1 tumour size (cm)	...	x_n patient's age (years)	malignant?
$i = 1$	10		52	1
$i = 2$	2		73	0
$\vdots$	$\vdots$		$\vdots$	$\vdots$
$i = m$	11			

– **Training examples:** $i = 1, \dots, m$

– **Features:** $j = 1, \dots, n$

– **Hypothesis Function:**

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

Note: But the squared error cost function,

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2$$

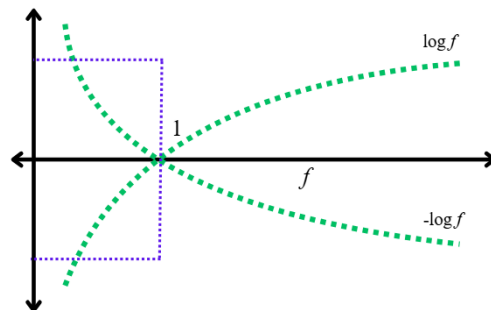
will not work, since gradient descent cannot be used in Logistic settings (convexity of the Cost function with varying parameters is not guaranteed)

Logistic Cost Function

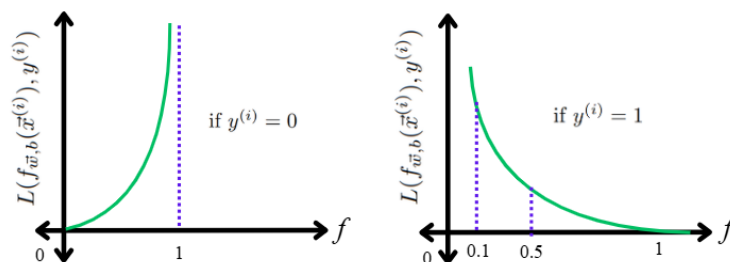
$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

where $L(\dots)$ measures the loss between $\hat{y}$ and y .

Notice how the Logarithmic curve looks like,



If we zoom into the 2 rectangles, we get



As $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 1$, The Loss $\rightarrow 0$

As $f_{\vec{w},b}(\vec{x}^{(i)}) \rightarrow 0$, The Loss $\rightarrow \infty$

Note: Since, this new Cost function is convex, we can use gradient descent again.

Deriving the Cost Function

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

The above piecewise function can be absorbed into a single formula, where the value of $y^{(i)}$ will cancel the other term,

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$$

if $y^{(i)} = 1$

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$$

if $y^{(i)} = 0$

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - 0) \cdot \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$$

So,

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m [L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})]$$

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))]$$

Note: This derivation involves the knowledge of maximum likelihood principles from statistics.

Using Gradient Descent on $J(\vec{w}, b)$

repeat until convergence:

$$\begin{cases} w \triangleq w - \alpha \cdot \frac{\partial}{\partial w} J(w, b) \\ b \triangleq b - \alpha \cdot \frac{\partial}{\partial b} J(w, b) \end{cases}$$

simultaneous updates.

For $J(\vec{w}, b)$,

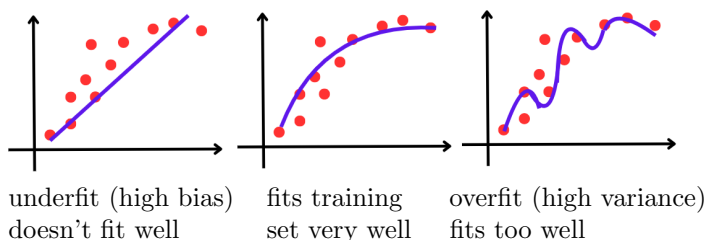
$$\frac{\partial}{\partial w} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

$$\frac{\partial}{\partial b} J(w, b) = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

Note: Yes ! These update terms look like that of Linear Regression but it is obviously different as the definition of f (hypothesis function) has changed from a linear to a logistic one.

Fitting Issues

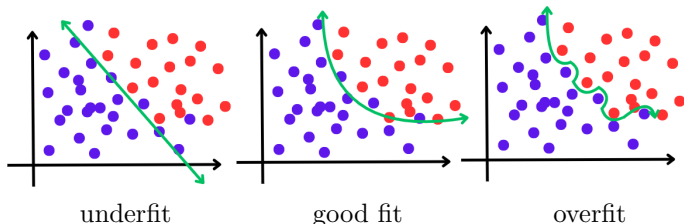
Example:



Note: The problem with over fitting is that the model will perform really well on the training dataset, but will not generalise well or perform poorly on test (validation) datasets.

Note: We can have an analogous graphic for classification problems with decision boundaries instead of regression as above,

Example:



Some ways to solve overfitting,

- Collect more data
- Can you use fewer features (including/exchanging features)
- w_j shouldn't have high magnitudes, can be eliminated by replacing with 0
- use regularisation

Note: Too many features in a model relative to number of training examples is bad, consider simplifying

Regularisation

What we can do is penalise terms in the cost function by associating large coefficients (will cause small values of w_j) effectively making it vanish and giving us a simpler model.

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \boxed{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}$$

where the boxed term is called the **regularisation term** and $\lambda > 0$ is the **regularisation parameter**.

Note: Out of all w_j , b , we don't penalise b . Some people include this in the formula but this makes very little difference,

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 + \frac{\lambda}{2m} b^2$$

We now have a new goal,

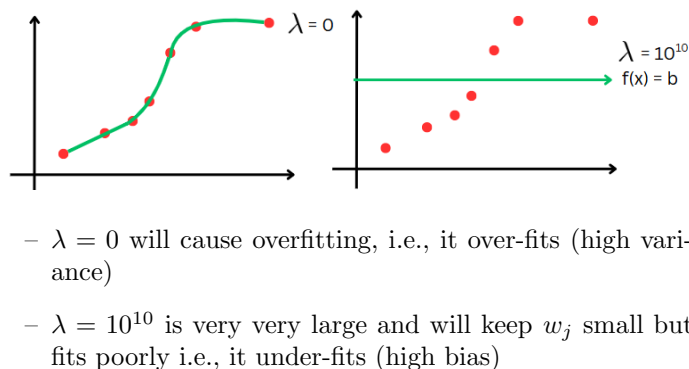
$$\min_{\vec{w}, b} J(\vec{w}, b) = \min_{\vec{w}, b} \left[\frac{1}{2m} \sum_{i=1}^m \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 \right]$$

where $\lambda > 0$.

With our value for λ we are trying to tradeoff/balance 2 things:

- fit data
- make w_j small

Example:



Regularised Gradient Descent

repeat {

$$w_j \triangleq w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \cdot x_j^{(i)} + \frac{\lambda}{m} w_j \right]$$

$$b \triangleq b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

simultaneously update:

$$w_j \quad \forall j = 1, \dots, n \text{ \& } b$$

}

Example:

Intuition for why this works ?

If we pull out the w_j update term, we get,

$$w_j \triangleq \boxed{1 \cdot w_j - \alpha \frac{\lambda}{m} \cdot w_j} - \boxed{\alpha \frac{1}{m} \sum_{i=1}^m \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}}$$

where the first boxed term is $w_j \left(1 - \alpha \frac{\lambda}{m}\right)$ and the second term is the usual update term for unregularised gradient descent.

For the first term, as $\alpha \rightarrow 0.01$ and $\frac{\lambda}{m} = \frac{1}{50} = 0.02$

we get, $(1 - 0.0002) = 0.9998$ (shrinking w_j on every iteration)

Derivation of the Derivative Term

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{\partial}{\partial w_j} \left[\frac{1}{2m} \sum_{i=1}^m \left(f(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 \right]$$

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m \left((\vec{w} \cdot \vec{x} + b - y^{(i)}) \cdot x_j^{(i)} \right) + \frac{\lambda}{2m} \cdot 2w_j$$

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \left((\vec{w} \cdot \vec{x} + b - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{m} w_j$$

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m \left((f(\vec{x}) - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{m} w_j$$

Regularisation of Logistic Regression

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$
